CG3207 Assignment 3 — Study Notes & Demo Guide

Multiplication and division in the MCycle unit, radix-2 Booth, and putting it all into the RISC-V CPU

Part I prepares you for the demo: what to expect, a demo script, and our slides explained. Part II goes from the basics up through every task and every code change, then a question bank, glossary, submission checklist and a one-page cheat sheet. Every worked example was traced with a script that copies the hardware step by step (and the Booth traces were checked against a Verilog simulation), so the bit patterns are exactly what the circuit produces.

Contents

§	Section
A	**The demo: what to expect, checklist, demo script**
B	**Our slides, explained slide by slide**
0	The big picture: which files changed
1	Background: two's complement and sign extension
2	How the MCycle unit works (the template's machinery)
3	The original multiplier (shift-and-add)
4	Task 1: restoring division, unsigned then signed
5	Task 3: radix-2 Booth multiplication — the algorithm, proof, code and worked examples
5B	Radix-4 Booth — idea, table, proof, hardware, Verilog, worked examples, radix-2 vs radix-4 side by side
5C	Practice problems with answers (radix-2 and radix-4)
5D	Other enhancement options
6	Task 2: putting MCycle into the CPU
7	The demo program
8	How everything was tested
8B	Task 4: the visualisation, and the division walkthrough
9	Questions you're likely to be asked, with answers
10	Glossary
11	Submission checklist
12	One-page cheat sheet

PART I — THE DEMO

A. What the demo is like, and how to run it

A.1 What to expect

From another group that has already demoed Assignment 3 (second-hand, not official):

- **No debugging.** You don't fix anything live; you show a working design.
- **"Else all same"**: the format is the same as the earlier demos. You run your program on the board, explain it, and show the required files and simulations.
- **You will be quizzed on Booth.** That group did radix-4 and was told to "know radix-4 Booth to heart". The quiz follows your own design, so for us it will be **mostly radix-2** (Section 5): the rule, why it works, the registers, and a trace by hand. Expect radix-4 as the follow-up ("how would you make it faster?", "what changes for radix-4?"), so know Section 5B too. Section 5C has practice problems for both.
- **Everyone is scored individually**, so every member must be able to answer any question, including about parts they didn't write.

The manual also says: only **one assembly program and one bitstream** for the whole demo, and the files must be **exactly** the ones submitted.

A.2 Before the demo — checklist

1. **Bitstream rebuilt** with the final MCycle.v (radix-2 Booth, `BOOTH = 1`), `TOP` as top, `TOP_Nexys4DDR.xdc` enabled. Program the board once beforehand and check the LEDs show 0xFF and view 9 shows `00000021`.
2. **Open and ready on the laptop:**
   - Vivado with the project; `test_MCycle` and `test_Wrapper` behavioural simulations already run (`run all`), output visible: **PASSED - all 76083 checks** and **229 passed, 0 failed**.
   - The post-synthesis simulation result and utilisation report for both versions (BOOTH = 1 and 0), for the slides' numbers.
   - The slides (Task 3).
   - `booth_visualisation.html` in a browser (Task 4).
   - `MCycle.v` open in an editor, ready to point at the Booth block (lines 203–225) and the division block (lines 243–264).
3. **Submission uploaded** before your lab slot (Section 11).

A.3 A demo script (about 5–8 minutes)

Step	Show	Say
1	Board after programming: **LED7–0 all on**	"At reset the program runs mul, mulh, mulhu, div, divu, rem and remu on 16 corner cases and checks every result against a table. Each LED is one instruction; all on means all 7 × 16 results are right."
2	SW15 = 0, SW3–0 = **1**, SW7–4 = **2** → `FFFFFFEB`	"Case 1 is −7 × 3 = −21, computed live by the `mul` instruction."
3	SW3–0 = **4**, SW7–4 = **5** → `80000000`	"Case 4 is $-2^{31} \div -1$, the overflow case. RISC-V says the answer is -2^{31} with remainder 0, and that's what we get."
4	SW3–0 = **9**, SW7–4 = **5** → `FFFFFFFF`; SW7–4 = **7** → `00000005`	"5 ÷ 0: the manual lets us assume this never happens, but we still give the RISC-V answer: quotient all ones, remainder = dividend."
5	SW7–4 = **9** → `00000021`	"This is the hardware cycle counter measuring one `mul`: 33 cycles. With the original signed multiplier it was 65."
6	SW15 = 1, e.g. a = −5 (SW14–8 = 1111011), b = 3 (SW6–0 = 0000011) → `FFFFFFF1`; hold BTNL → `FFFFFFFF`; BTNR → `55555553`	"Own operands: × by default, ÷ / rem / divu with the buttons. Note divu treats −5 as 4 294 967 291." Let the TA pick numbers.
7	Slides 2–5	Section B.
8	`booth_visualisation.html` with the TA's numbers	Step through 4-bit −3 × 5 edge by edge (Section 8B).
9	Simulations	`test_MCycle` and `test_Wrapper` pass; post-synthesis simulation of both versions passes.

Two different cycle conventions — know both. The slides count **computation iterations**: 64 → 32. The board and testbenches count **cycles including the write-back cycle** (the cycle in which Busy falls and the register is written): 65 → 33. Both are correct; say which you mean. (Section 2.7.)

B. Our slides, explained slide by slide

The deck has four content slides, numbered 2–5.

Slide 2 — "Radix-2 Booth halves signed multiplication cycles"

What it says: the baseline sign-extends both operands to 64 bits and does 64 shift-and-add iterations, one bit of the *extended* multiplier per iteration. Booth handles the signed 32-bit multiplier directly, in 32 add/subtract-and-shift iterations, one *original* multiplier bit per iteration. Result: 50% fewer computation cycles for 32-bit signed multiplication. Unsigned multiplication remains at 32 cycles, and no `*` operator is used.

What it means:

- The baseline can only *add* shifted copies of the multiplicand, so it can't directly handle a negative multiplier, whose top bit is worth -2^{31}. Its trick is to sign-extend to 64 bits and multiply modulo 2^{64} (Section 3.3). That doubles the bits to process: 64 iterations.

- Booth recoding turns the signed multiplier into digits {−1, 0, +1}, including the negative weight of the sign bit, so it only needs the 32 real bits (Section 5.3).
- "Unsigned multiplication remains at 32 cycles": `mulhu` still goes through the original shift-and-add (MCycleOp = 01), which already needed only 32 iterations because unsigned numbers don't need sign extension.
- "No `*` operator": the manual forbids `*` because it would infer the FPGA's DSP multiplier blocks. The DSP count of 0 on slide 5 proves it.

Likely follow-ups: "Why does the baseline need 64 iterations?" "Why doesn't Booth need sign extension?" "Why didn't you use Booth for unsigned too?" (Section 9.)

Slide 3 — "How radix-2 Booth updates the hardware"

What it says: the registers (A 33 bits accumulator; M 33 bits sign-extended multiplicand; Q 32 bits multiplier, then lower product bits; Q_{-1} 1 bit, the previous least-significant multiplier bit), the loop (inspect {Q[0], Q_{-1}} → add/subtract/hold → arithmetic right shift → repeat 32 times), the rule table (00/11 keep A; 01 A ← A + M; 10 A ← A − M), the result {A[31:0], Q}, and why A and M have 33 bits.

What it means: this is the whole algorithm. Section 5.4 explains each register, 5.5 explains each line of the Verilog, and 5.6 explains the 33 bits. The key phrases to be able to say from memory:

- "01 means a run of 1s just ended, so add M; 10 means a run of 1s starts, so subtract M; 00 and 11 mean we're inside a run, so do nothing."
- "The shift is arithmetic because A is a signed partial product."
- "The extra bit is so that −M fits when M is -2^{31}."
- "Q starts as the multiplier and ends as the low half of the product: each shift moves one finished product bit in from A and one used multiplier bit out to Q_{-1}."

Slide 4 — "Verified against expected products and the baseline"

What it says: the worked example −3 × 5 with 4-bit operands, iteration by iteration (A − M, A + M, A − M, A + M; final {A[3:0], Q} = 1111 0001 = 0xF1 = −15), and a verification table: 4-bit comparison 514 checks including every signed operand pair; 32-bit behavioural 467 checks per design; 32-bit post-synthesis functional 467 checks per design.

What it means: the worked example is exactly the trace in Section 5.7. Practise it on paper until you can do it without looking; it's the most likely "show me" question.

The verification numbers come from **Hin Lok's own testbench**. The repository's `test_MCycle.v` is a separate, larger self-checking testbench: **76,083 checks** (every operand pair at 4 and 6 bits for all four operations, 32-bit corners + 3,000 random per operation, cycle counts, Busy timing, operand changes while busy, reset mid-operation, both BOOTH settings), plus `test_MCycle_postsynth` on the netlist. Since the testbench you submit should be the one behind the slide's numbers, either submit Hin Lok's testbench too, or update the slide to quote `test_MCycle.v`'s numbers (or both).

Slide 5 — "Fewer cycles at a higher hardware cost"

What it says (Vivado 2023.2, Artix-7 XC7A100T, 32-bit, 10 ns clock constraint):

Metric	Baseline	Booth	Change
Signed multiplication cycles	64	32	−50%
Slice LUTs	487	650	+33.5%
Flip-flops	266	365	+37.2%
DSP blocks	0	0	unchanged
Inferred latches	0	0	none
Worst setup slack	+0.442 ns	+0.496 ns	
Worst hold slack	−1.079 ns	−1.117 ns	

What it means, line by line:

- **Cycles −50%:** 64 → 32 iterations (65 → 33 including write-back).
- **Flip-flops +99:** exactly the Booth registers: A (33) + M (33) + Q (32) + Q_{-1} (1) = **99 bits**. If asked "where do the extra flip-flops come from?", this is the precise answer.
- **LUTs +163:** the 33-bit adder/subtractor for A ± M, the 3-way choice (add / subtract / hold), and the multiplexers that load and shift A, Q and Q_{-1}. The baseline's 64-bit adder and shift registers are still there, because unsigned multiply (`mulhu`) still uses them.
- **DSP 0:** no `*` was used.
- **Latches 0:** every combinational signal has a default value in the `always @(*)` block, so nothing is inferred as a latch (a latch is memory created by accident when a combinational signal isn't assigned on every path).
- **Setup slack +0.4 ns:** at a 10 ns (100 MHz) clock, the slowest path still arrives 0.4–0.5 ns before the clock edge, so both designs meet timing at 100 MHz (post-synthesis estimate). Booth's critical path is slightly shorter, plausibly because its adder is 33 bits instead of 64, but the difference is small.
- **Hold slack −1.1 ns:** negative hold slack *after synthesis* is normal. Synthesis timing uses an ideal clock with no routing delays; hold problems are fixed in implementation (place-and-route adds delay where needed). It's the same in both designs, so it isn't caused by Booth. If asked, say "post-synthesis hold numbers aren't meaningful; place-and-route fixes hold, and the post-implementation timing report is what counts." (On the board the CPU also runs on a divided clock, 3.125 MHz, far below the 100 MHz constraint.)
- **Trade-off:** 50% fewer signed-multiply cycles for 33.5% more LUTs and 37.2% more flip-flops. The manual explicitly accepts "the trade-off between the two".

Two things to double-check before the demo:

1. **Which MCycle.v was measured?** Hin Lok measured his version, in which division was still the template's 1-cycle placeholder. The submitted MCycle.v has the real division, which adds the same division hardware to both baseline and Booth. So the *differences* (+163 LUTs, +99 FFs) still hold, but the *percentages* will be smaller on the final file. Either re-run Vivado on the final MCycle.v with `-generic BOOTH=1` and `BOOTH=0` (Section 8.3), or add a footnote: "measured with the division placeholder, to isolate the multiplier". A TA who synthesises your submitted file will otherwise see different numbers.
2. **"Unsigned remains at 32 cycles"** is in iteration terms (33 including write-back), consistent with the 64 → 32 convention.

PART II — THEORY AND CODE

0. The big picture

File	Task	What changed
`MCycle.v`	1, 3	Added division (Task 1); radix-2 Booth for signed multiply (Task 3, Hin Lok's design)
`test_MCycle.v`	1	Turned into a self-checking testbench; added a post-synthesis version
`Decoder.v`	2	Recognises the M instructions and generates MCycle's control signals
`RV.v`	2	Instantiates MCycle, stalls the CPU while it's busy, selects its result
`Wrapper.v`	2	Instruction and data memories enlarged from 128 to 256 words
`test.asm`, `AA_IROM.mem`, `AA_DMEM.mem`	2	New demo program
`test_Wrapper.v`	2	New self-checking testbench for the program
`cpu_check.py`	2	Updated so the Assignment 2 regression test knows about MCycle
`booth_visualisation.html`	4	Interactive step-by-step visualisation of the Booth multiplier
Booth slides (PDF)	3	1–4 slides on the enhancement (Section B)
`prompt_history.md`	4	Every AI prompt used for the visualisation

Code map of the final `MCycle.v` (handy when a TA says "show me where..."):

Lines	What
65–66	Parameters: `width` (4 for unit tests, 32 in the CPU), `BOOTH` (1 = radix-2 Booth, 0 = original)
87–88	Last iteration numbers: `width − 1` (Booth, unsigned mul, division), `2·width − 1` (original signed mul)
130–151	`IDLE_PROCESS`: the IDLE/COMPUTING state machine and Busy (Section 2.2)
154–176	`COMPUTING_PROCESS` begins: hold defaults for every `n_` variable, intermediate defaults (no latches), operand signs and magnitudes for division
182–199	Start of an operation: load the template registers, the Booth registers (187–190) and the division registers and sign flags (193–198)
202	The guard: only iterate while still computing (Section 2.5)
203–225	**Radix-2 Booth signed multiply** (Section 5.5)
226–242	Original shift-and-add: unsigned multiply (and signed when BOOTH = 0)
243–264	**Restoring division**, signed and unsigned (Section 4)
269–284	`STATE_UPDATE_PROCESS`: every register updated at the clock edge

In `RV.v`: line 142 `WE_PC = ~Busy`, 167 `WE = RegWrite & ~Busy`, 168–169 the MCycle result muxes, 267 the MCycle instance. In `Decoder.v`: line 77 `FUNCT7_M`, 97–120 the R-type / M-instruction decoding. | `docs/CG3207_Task1_Division_Dataflow.html` | 1 | Study aid: one division traced signal by signal through MCycle.v (Qi Ye) |

---

1. Background: two's complement and sign extension

Every signed number in the CPU is in **two's complement**. In a w-bit number:

- The top bit has weight $\mathbf{-2^{w-1}}$.
- Every other bit i has its normal weight $\mathbf{+2^{i}}$.

For 4 bits:

Bits	Unsigned	Signed
0111	7	7 (largest positive)
1000	8	−8 (most negative)
1101	13	−8 + 4 + 1 = −3
1111	15	−1

Three facts are used everywhere in this assignment.

Fact 1: negation is "invert all bits, then add 1". −(0011) = 1100 + 1 = 1101 = −3. The hardware can therefore subtract X by adding the inverted X with a carry-in of 1. That's how synthesis builds the Booth multiplier's A − M.

Examples on 32 bits:

Value	Step	Hex
1		`00000001`
	invert	`FFFFFFFE`
−1	+1	**`FFFFFFFF`**
15		`0000000F`
	invert	`FFFFFFF0`
−15	+1	**`FFFFFFF1`**

To read a negative number, negate it back. `FFFFFFEE` → invert → `00000011` → +1 → `00000012` = 18, so `FFFFFFEE` = −18. A quick shortcut for small negatives: `FFFFFFFF − FFFFFFEE = 0x11 = 17`, so it is −1 − 17 = −18.

Fact 2: sign extension preserves the value. To widen a signed number, copy its top bit into the new positions: 1101 (−3) becomes 11111101, which is still −3 in 8 bits. To widen an unsigned number, fill with zeros: 1101 (13) becomes 00001101. Both multipliers do this, and MCycleOp[0] picks which (0 = signed, so copy the sign; 1 = unsigned, so fill with 0).

Fact 3: the most negative number has no positive partner. In 4 bits the range is −8 … +7, so −(−8) doesn't fit as a signed number: invert 1000 to get 0111, add 1, and you get 1000 again. It does fit as an unsigned number (1000 = 8). This matters for division (Section 4.7).

Arithmetic shift right (written `>>>`) shifts right and copies the sign bit into the top, so it divides a signed number by 2 rounding down: 1100 (−4) >>> 1 = 1110 (−2). The Booth multiplier shifts its {A, Q, Q_{-1}} register this way.

Signed vs unsigned meaning of the same bits. The same bits mean different numbers depending on whether an instruction treats them as signed or unsigned:

Bits	Signed (`div`, `mul`, `mulh`)	Unsigned (`divu`, `mulhu`)
`FFFFFFFF`	−1	4,294,967,295
`FFFFFFFA`	−6	4,294,967,290
`80000000`	−2,147,483,648	2,147,483,648

Example: −6 ÷ 3 with `div` gives −2 (`FFFFFFFE`). With `divu`, the bits `FFFFFFFA` are read as 4,294,967,290, and 4,294,967,290 ÷ 3 = 1,431,655,763 remainder 1 = **`55555553`**. Check: `0x55555553 × 3 = 0xFFFFFFF9`, + 1 = `0xFFFFFFFA` ✓. (It looks like a pattern because `FFFFFFFF ÷ 3 = 55555555` exactly, since `5 × 3 = F` in every hex digit; −5 with `divu` gives the same quotient with remainder 2.)

---

2. How the MCycle unit works (the template's machinery)

Before the algorithms, you need to know how MCycle runs any operation over several cycles. This machinery was kept from the template, so it's worth understanding properly.

2.1 The ports

```
input  Start          // "please start an operation" (from the Decoder)
input  [1:0] MCycleOp // 00 signed mul, 01 unsigned mul, 10 signed div, 11 unsigned div
input  Operand1, Operand2
output Result1        // low word of product / quotient
output Result2        // high word of product / remainder
output Busy           // 1 while working
```

2.2 The state machine (`IDLE_PROCESS`)

There are only two states: **IDLE** and **COMPUTING**.

```
IDLE      --Start=1-->  COMPUTING   (Busy = 1 immediately)
COMPUTING --done=0-->   COMPUTING   (Busy = 1)
COMPUTING --done=1-->   IDLE        (Busy = 0)
```

`Busy` is computed **combinationally**, from the current state and the inputs. In IDLE, Busy goes to 1 *in the same cycle* that Start rises, without waiting for a clock edge. A machine whose output depends on its inputs like this is a **Mealy machine**. It's essential for the CPU: the instant a `mul` instruction appears, Busy goes high and freezes the PC before the next clock edge can advance it.

2.3 The "next-value" coding style (`COMPUTING_PROCESS`)

Every register `x` has a partner `n_x`, meaning "what x will become at the next clock edge". The combinational block:

1. starts with `n_x = x` for every register (hold by default);
2. modifies the `n_` values step by step, like a little program;
3. the clocked block copies every `n_x` into `x` at the clock edge, in one place.

```
always @(posedge CLK) begin
    state <= n_state ; count <= n_count ; temp_sum <= n_temp_sum ; ...
end
```

The benefit is that several steps happen in one cycle, in order, inside one combinational block. For example, the start cycle first *loads* the operands into `n_` variables, and the very next lines *already run the first iteration* on those loaded values. The hardware is a chain of logic: load-mux → adder → shift → register input.

2.4 The start condition

```
if( RESET | (n_state == COMPUTING & state == IDLE) ) begin
    n_count = 0 ; n_temp_sum = 0 ;
    n_shifted_op1 = ... Operand1 ... ;  // capture the operands
    ...
end
```

`n_state == COMPUTING & state == IDLE` means "we are in IDLE and about to start". This is the only moment Operand1 and Operand2 are read. After that, MCycle works only on its own registered copies. That's important in the CPU (Section 6.6).

2.5 The guard

```
if( n_state == COMPUTING ) begin  ... one iteration ...  end
```

An iteration only runs if we will still be computing after this edge. In the cycle where Busy falls (state = COMPUTING, done = 1, so n_state = IDLE), no iteration runs and the results are held. That's the cycle in which the CPU writes the result to the register file, so the results must not change during it.

2.6 `count` and `done`

`count` counts iterations. When the last iteration runs, the code sets `n_done = 1`. `done` is registered, so it becomes 1 at the next edge, and then Busy falls.

2.7 Cycle-by-cycle timeline (32-bit Booth multiply, 32 iterations)

Cycle	state	Busy	What happens
1	IDLE	1 (because Start = 1)	Operands loaded **and** iteration 0 computed; stored at the edge
2	COMPUTING	1	iteration 1
…	COMPUTING	1	…
32	COMPUTING	1	iteration 31 (the last), `n_done = 1`, final results stored
33	COMPUTING, done = 1	**0**	no iteration; results stable; the CPU writes rd and the PC moves on
34	IDLE	depends on the next instruction	ready again

So the latency is **iterations + 1**: 32 + 1 = 33 for Booth multiply and for division, and 64 + 1 = 65 for the original signed multiply. That +1 is why the board measures 33, not 32.

---

3. The original multiplier (shift-and-add)

You need this to see what the enhancement improves.

3.1 Pencil-and-paper binary multiplication

Multiplication is repeated addition of shifted copies. In binary each multiplier bit is 0 or 1, so each step either adds a shifted copy of the multiplicand or adds nothing.

Example, unsigned 13 × 6 (4-bit: 1101 × 0110):

```
    1101      (13, multiplicand)
  x 0110      (6, multiplier)
  ------
    0000      bit 0 = 0 -> add nothing
   1101       bit 1 = 1 -> add 13 shifted left 1
  1101        bit 2 = 1 -> add 13 shifted left 2
 0000         bit 3 = 0 -> add nothing
--------
01001110      = 78 = 0x4E
```

3.2 How the template's hardware does it

- `shifted_op1` holds the multiplicand and shifts **left** 1 each cycle.
- `shifted_op2` holds the multiplier and shifts **right** 1 each cycle, so the bit currently in use is always bit 0.
- `temp_sum` accumulates the result.

```
if( n_shifted_op2[0] ) n_temp_sum = n_temp_sum + n_shifted_op1 ;  // add if bit = 1
n_shifted_op2 = {1'b0, n_shifted_op2[2*width-1:1]} ;              // next bit to bit 0
n_shifted_op1 = {n_shifted_op1[2*width-2:0], 1'b0} ;              // multiplicand x 2
```

One multiplier bit per cycle, so **w cycles** for unsigned.

3.3 How it handles signed numbers, and why that costs 2w cycles

The template sign-extends both operands to 2w bits and multiplies them as if they were unsigned 2w-bit numbers, keeping only the low 2w bits.

This works because the low 2w bits of a product only depend on the low 2w bits of the inputs (arithmetic is "modulo 2^{2w}"), and a sign-extended number is congruent to its signed value modulo 2^{2w}.

Example, -3×6:

```
11111101 (-3) x 00000110 (6)
= 11111101<<1 + 11111101<<2 = 11111010 + 11110100 = 1 11101110
low 8 bits = 11101110 = 0xEE = -18
```

The cost: the multiplier now has **2w bits**. For a negative multiplier, the extra w bits are all 1s and each needs an addition, so the loop must run **2w = 64 cycles** for 32-bit signed. It also needs a **2w-bit adder** (64-bit) and two 2w-bit shift registers. That's the weak spot Task 3 attacks.

---

4. Task 1: restoring division

4.1 Long division, decimal first

Dividing 13 by 3 by hand: bring down one digit at a time, ask "how many times does 3 go into what I have?", write that digit of the quotient, subtract, and carry the remainder forward.

4.2 In binary the question is only "yes or no"

In binary each quotient digit is 0 or 1, so each step is one question: **is the current remainder ≥ the divisor?**

- If yes, the quotient bit is 1 and you subtract.
- If no, the quotient bit is 0 and you leave the remainder alone.

4.3 The register layout

MCycle puts the remainder **R** and the dividend/quotient **Q** side by side in one 2w-bit register (`temp_sum`):

```
temp_sum = { R (upper w bits) , Q (lower w bits) }
```

- At the start: R = 0, Q = dividend, and D (the divisor) is kept in `shifted_op2`.
- Each cycle:
  1. **Shift {R, Q} left by 1.** The top bit of Q (the next dividend bit, "brought down") moves into R. A free slot appears at the bottom of Q.
  2. **Trial subtract:** compute R − D.
  3. If R − D ≥ 0, keep the difference as the new R and put **1** in the free slot.
  4. Otherwise, keep R unchanged ("restore") and put **0** in the free slot.

After w cycles, all the dividend bits have moved into R and been replaced by quotient bits. Q now holds the **quotient** and R holds the **remainder**.

It's called *restoring* division because when the trial subtraction fails, the original R is kept, i.e. restored.

4.4 Worked example: 13 ÷ 3 in 4 bits

Start: R = 0000, Q = 1101 (13), D = 0011 (3).

Cycle	After shift: R (5 bits)	Q	R − D	Decision	New R	New Q
0	00001	101_	1 − 3 = −2	restore, q = 0	0001	1010
1	00011	010_	3 − 3 = 0	keep, q = 1	0000	0101
2	00000	101_	0 − 3 = −3	restore, q = 0	0000	1010
3	00001	010_	1 − 3 = −2	restore, q = 0	0001	0100

Result: Q = 0100 = **4**, R = 0001 = **1**. Check: 4 × 3 + 1 = 13 ✓.

Follow the quotient bits as they enter at the right: 0, 1, 0, 0, giving 0100.

4.5 Why R needs one extra bit

After the shift, R = 2 × (old R) + (new bit). Old R is always less than D, so the new R is at most 2D − 1. If D is large (say D = 1111 = 15), the shifted R can be up to 29, which needs **5 bits** in a 4-bit design. If the top bit were thrown away, the comparison would be wrong.

So the shift keeps the bit that falls out:

```
div_shifted = { n_temp_sum, 1'b0 } ;   // 2w+1 bits: the extra top bit is kept
```

The trial subtraction is done in **w + 2 bits**, so the top bit is a reliable sign (negative means R < D):

```
div_diff = { 1'b0, div_shifted[2*width : width] } - { 2'b00, n_shifted_op2[width-1:0] } ;
if( ~div_diff[width+1] )   // not negative -> R >= D
    n_temp_sum = { div_diff[width-1:0], div_shifted[width-1:1], 1'b1 } ;   // keep, q = 1
else
    n_temp_sum = { div_shifted[2*width-1:1], 1'b0 } ;                       // restore, q = 0
```

When R ≥ D, the difference is always less than D, so it fits back into w bits. When R < D, R itself is less than D and also fits. So R only needs w bits *between* cycles; the extra bit is only needed during the comparison.

This is 1 quotient bit per cycle, so **w = 32 iterations + 1 = 33 cycles**. Unsigned division (`divu`) is exactly this, with no further steps.

4.6 Signed division: magnitudes, then fix the signs

Restoring division only works on non-negative numbers. For signed division:

1. **At the start:** take the magnitudes, |dividend| and |divisor|, and remember two signs:
   - **Quotient sign:** negative if the operand signs differ (− ÷ + or + ÷ −).
   - **Remainder sign:** the same as the **dividend's** sign.
2. **Divide** the magnitudes with the unsigned algorithm above.
3. **At the end:** negate the quotient and/or the remainder as remembered.

```
op1_neg = ~MCycleOp[0] & Operand1[width-1] ;      // only signed division looks at signs
op2_neg = ~MCycleOp[0] & Operand2[width-1] ;
dividend_mag = op1_neg ? -Operand1 : Operand1 ;
divisor_mag  = op2_neg ? -Operand2 : Operand2 ;
...
n_neg_quotient  = (op1_neg ^ op2_neg) & (|Operand2) ;
n_neg_remainder = op1_neg ;
...
n_Result1 = n_neg_quotient  ? -quotient  : quotient ;
n_Result2 = n_neg_remainder ? -remainder : remainder ;
```

Why these sign rules? RISC-V (like C) defines signed division as **rounding towards zero**, and it requires dividend = quotient × divisor + remainder.

a ÷ b	magnitudes	quotient	remainder	check
7 ÷ 3	7 ÷ 3 = 2 r 1	2	1	2×3+1 = 7
−7 ÷ 3	2 r 1	**−2** (signs differ)	**−1** (dividend negative)	−2×3−1 = −7
7 ÷ −3	2 r 1	**−2**	**1** (dividend positive)	−2×−3+1 = 7
−7 ÷ −3	2 r 1	**2** (signs equal)	**−1**	2×−3−1 = −7

These are your test cases 0–3, and the board shows exactly these values.

4.7 The edge cases

$-2^{w-1} \div -1$ (overflow). The true answer $+2^{31}$ doesn't fit in 32-bit signed. RISC-V says the result must be -2^{31} with remainder 0. Here's what the hardware does with 4 bits (−8 ÷ −1):

- |−8| = −(1000) = 1000. As an unsigned magnitude that's 8 (Fact 3), so it's fine.
- 8 ÷ 1 = 8 = 1000, remainder 0.
- The signs are equal, so the quotient isn't negated. Result: 1000, which read as signed is −8.

So the required RISC-V answer comes out naturally, with no special-case hardware. That's test case 4 (`80000000`).

Division by zero. The manual says you can assume it never happens. Still, it's worth knowing what the hardware does:

- With D = 0, every trial subtraction R − 0 ≥ 0 succeeds, so every quotient bit is 1. The quotient is all ones, and R ends as the dividend.
- RISC-V specifies exactly that: quotient = all ones (−1), remainder = dividend.
- For signed division, the `& (|Operand2)` term stops the quotient from being negated when the divisor is 0, so −5 ÷ 0 also gives −1, not +1. `|Operand2` is OR of all bits, which is 1 if the divisor is non-zero.

That's why case 9 (5 ÷ 0) shows `FFFFFFFF` and remainder 5, matching RARS.

---

5. Task 3: radix-2 Booth multiplication

The group's Task 3 enhancement is Hin Lok's radix-2 Booth multiplier for **signed** multiplication (MCycleOp = 00). This is the most important section for the demo quiz. (An earlier radix-4 version is in the git history at commit `6c27ed1`; radix-4 itself is explained in Section 5B.)

5.1 The problem Booth solves

The original multiplier (Section 3) can only add **positive** shifted copies of the multiplicand: each multiplier bit is 0 or 1, and bit i is worth $+2^i$. But in a signed multiplier the top bit is worth $\mathbf{-2^{w-1}}$ (Section 1). The template's workaround is to sign-extend both operands to 2w bits and multiply modulo 2^{2w}, which needs **2w = 64 iterations** and a **64-bit adder**.

Booth's algorithm rewrites the multiplier so that the negative weight of the sign bit is handled directly. It processes only the **w = 32 real bits**: 64 → 32 iterations (65 → 33 cycles with write-back).

5.2 The idea: replace a run of 1s by a subtraction and an addition

A block of consecutive 1s is a difference of two powers of two:

```
0011 1100  =  0100 0000 - 0000 0100        (60 = 64 - 4)
```

So instead of adding M once for every 1 in the run (4 additions), you can **subtract M at the start of the run** (its lowest 1, weight 4) and **add M just after its end** (weight 64). Inside a run of 1s or a run of 0s, nothing needs to happen.

The hardware finds the start and end of runs by looking at **two adjacent bits**: the current bit Q_0 and the bit to its right, Q_{-1} (an imaginary 0 to the right of the lowest bit at the start). Scanning from right (LSB) to left:

$Q_0\ Q_{-1}$	Where are we?	A becomes
0 0	inside a run of 0s	A (unchanged)
0 1	the bit to the right was 1, this one is 0: **a run of 1s just ended**	A + M
1 0	this bit is 1, the one to the right was 0: **a run of 1s starts**	A − M
1 1	inside a run of 1s	A (unchanged)

Memory aid: **"10 = start of 1s = subtract; 01 = end of 1s = add."**

5.3 Why it gives the right answer (the proof)

The rule says the digit used at bit position i is

$\mathbf{d_i = Q_{i-1} - Q_i}$ (with $Q_{-1} = 0$)

which is exactly the table: 01 → +1, 10 → −1, 00 and 11 → 0. The multiplier Booth "sees" is $\Sigma\ d_i \cdot 2^i$ for i = 0 … w−1. Expand it:

$\Sigma\ (Q_{i-1} - Q_i) \cdot 2^i$

Each bit Q_i (for $i \le w-2$) appears twice:

- once as $\mathbf{-Q_i \cdot 2^i}$ (in digit i), and
- once as $\mathbf{+Q_i \cdot 2^{i+1}}$ (in digit i+1, where it is the "previous" bit).

Together: $-Q_i \cdot 2^i + Q_i \cdot 2^{i+1} = \mathbf{+Q_i \cdot 2^i}$, its normal weight ✓.

The **top bit** Q_{w-1} only appears in digit w−1 (there is no digit w), so it keeps only $\mathbf{-Q_{w-1} \cdot 2^{w-1}}$. That is **exactly the two's complement weight of the sign bit** ✓.

So $\Sigma\, d_i \cdot 2^i$ equals the multiplier's signed value, and $M \times \Sigma\, d_i \cdot 2^i = \Sigma\, (d_i \cdot M) \cdot 2^i$ is the signed product. This is why Booth needs no sign extension: the sign bit's negative weight is built into the recoding.

Check with Q = 5 = 0101 (pairs (Q_i, Q_{i-1}) from the right): (1,0) → −1, (0,1) → +1, (1,0) → −1, (0,1) → +1. Value: $-1 \cdot 1 + 1 \cdot 2 - 1 \cdot 4 + 1 \cdot 8 = 5$ ✓.

Check with Q = −2 = 1110: (0,0) → 0, (1,0) → −1, (1,1) → 0, (1,1) → 0. Value: $-1 \cdot 2 = -2$ ✓. A negative number is one run of 1s reaching the top, so it needs only one subtraction and no matching addition, because the run never ends.

5.4 The hardware: four registers, one adder, one shifter

Register	Bits (w = 32)	Starts as	Role
M (`booth_M`)	w+1 = 33	multiplicand, sign-extended by 1 bit	the value added or subtracted; never changes
A (`booth_A`)	w+1 = 33	0	accumulator; ends as the upper half of the product
Q (`booth_Q`)	w = 32	multiplier	bits still to be read; ends as the lower half of the product
Q_{-1} (`booth_Qminus1`)	1	0	the bit most recently shifted out of Q

Each clock edge does three things in **one** combinational pass (they are drawn as steps, but they're one cycle):

1. **Decide:** look at $\{Q_0, Q_{-1}\}$.
2. **Add/subtract:** A = A + M (01), A − M (10), or unchanged (00/11). One 33-bit adder/subtractor.
3. **Shift:** shift the whole 66-bit $\{A, Q, Q_{-1}\}$ right by one bit, **copying A's sign bit** into the top (arithmetic shift). A's lowest bit moves into the top of Q; Q's lowest bit becomes the new Q_{-1}.

Why shift right instead of shifting M left? In pencil-and-paper multiplication you shift the multiplicand left for each new bit (weights 1, 2, 4, …). Shifting the *partial product* right by 1 each step has the same relative effect: each new contribution is added one position higher than the bits already shifted down. The benefit is that the adder only ever works on the top part (A, 33 bits) instead of a 64-bit sum. The finished low bits of the product drop out of A into Q, one per cycle, exactly as the used multiplier bits leave Q into Q_{-1}, so Q is "recycled" as product storage.

5.5 The Verilog, line by line (MCycle.v lines 203–225)

```
if( (MCycleOp == 2'b00) & (BOOTH != 0) ) begin
```

Only **signed** multiplication (op 00) uses Booth, and only when the `BOOTH` parameter is 1 (with 0 the original multiplier is used, for the before/after comparison).

```
    case( {n_booth_Q[0], n_booth_Qminus1} )
        2'b01:   n_booth_A = n_booth_A + n_booth_M ;
        2'b10:   n_booth_A = n_booth_A - n_booth_M ;
        default: ;
    endcase
```

The decision table. `n_booth_*` are the "next value" variables (Section 2.3). In the **first** cycle they were just loaded from the operands a few lines earlier (A = 0, M = {sign, Operand1}, Q = Operand2, Q_{-1} =

0, lines 187–190); in later cycles they hold the registered values. A − M is built by synthesis as A + ~M + 1 (Fact 1), so one adder does both.

```
    booth_shift = { n_booth_A, n_booth_Q, n_booth_Qminus1 } ;
    booth_shift = { booth_shift[2*width+1], booth_shift[2*width+1:1] } ;
```

Glue A, Q and Q_{-1} into one 2w+2 = 66-bit vector, then shift it right by 1, repeating the top bit (A's sign). The comment in the code notes this is written explicitly instead of using `>>>`, because `>>>` only sign-extends on a `signed` type and it's easy to get wrong in Verilog.

```
    n_booth_A = booth_shift[2*width+1:width+1] ;
    n_booth_Q = booth_shift[width:1] ;
    n_booth_Qminus1 = booth_shift[0] ;
```

Cut the shifted vector back into A (top 33 bits), Q (next 32) and Q_{-1} (bottom bit).

```
    if( n_count == COUNT_LAST_W[7:0] ) n_done = 1'b1 ;   // COUNT_LAST_W = width-1 = 31
    n_count = n_count + 1 ;
```

Iterations 0 … 31 = 32 iterations; on the last one, `done` is set so Busy falls at the next edge.

```
    n_Result2 = n_booth_A[width-1:0] ;   // high word
    n_Result1 = n_booth_Q ;              // low word
```

The product is {A[31:0], Q}. A's extra top bit (bit 32) is only a copy of the sign at the end, and Q_{-1} is just history, so both are discarded. Results are updated every cycle, but only the final values matter (the CPU reads them when Busy falls).

5.6 Why A and M have one extra bit

- **−M must fit.** If $M = -2^{31}$ (most negative), then $-M = +2^{31}$, which doesn't fit in 32-bit signed (Fact 3). In 33 bits it fits. In 4 bits: M = −8 = 1000; −M = +8 needs 5 bits: 01000.
- **A must not overflow.** A holds a running partial sum; adding or subtracting M can exceed the 32-bit range for one step before the shift halves it again. One extra bit is always enough in radix-2 Booth: each step adds or subtracts at most $|M| \le 2^{w-1}$ to a value that the previous shift has just halved, so A ± M stays within w+1 signed bits. The exhaustive 4- and 6-bit tests (every operand pair, including the most negative ones) confirm it.
- At the end, bit 32 of A is just the sign repeated, so the product is exactly {A[31:0], Q}.

`M = { Operand1[width-1], Operand1 }` (line 188) is that 1-bit sign extension.

5.7 Worked example 1: −3 × 5 in 4 bits (the slide 4 example)

M = −3 = **11101** (5 bits), so −M = +3 = **00011**. Q = 5 = **0101**, A = 00000, Q_{-1} = 0. Four iterations.

Edge	$\{Q_0, Q_{-1}\}$	Operation	A before shift	A after	Q after	Q_{-1} after
start	—	load	—	00000	0101	0
1	1 0	A − M = 0 + 3 = 3	00011	00001	1010	1
2	0 1	A + M = 1 − 3 = −2	11110	11111	0101	0
3	1 0	A − M = −1 + 3 = 2	00010	00001	0010	1
4	0 1	A + M = 1 − 3 = −2	11110	11111	0001	0

Doing edge 1's shift by hand: {A, Q, Q_{-1}} = 00011 0101 0 (10 bits). Shift right by 1, copying the sign (0): 0 00011 0101 → **00001 1010 1**. A's low bit (1) went into the top of Q, and Q's low bit (1) became Q_{-1}.

Edge 2: {A, Q, Q_{-1}} = 11110 1010 1 → shift, copying sign 1 → **11111 0101 0**.

Product = {A[3:0], Q} = **1111 0001 = 0xF1 = −15** ✓.

5.8 Worked example 2: 6 × −2 (positive M, negative multiplier)

M = 6 = **00110**, −M = **11010** (−6). Q = −2 = **1110**.

Edge	{Q_0, Q_{-1}}	Operation	A before shift	A after	Q after	Q_{-1} after
1	0 0	hold	00000	00000	0111	0
2	1 0	A − M = 0 − 6	11010	11101	0011	1
3	1 1	hold	11101	11110	1001	1
4	1 1	hold	11110	11111	0100	1

Product = 1111 0100 = 0xF4 = **−12** ✓. Only one subtraction: the run of 1s in 1110 starts at bit 1 and never ends, because it reaches the sign bit (Section 5.3).

5.9 Worked example 3: the hard case, −8 × −1

M = −8 = **11000**, −M = **01000** (+8, only possible because of the extra bit). Q = −1 = **1111**.

Edge	{Q_0, Q_{-1}}	Operation	A before shift	A after	Q after	Q_{-1} after
1	1 0	A − M = 0 + 8	01000	00100	0111	1
2	1 1	hold	00100	00010	0011	1
3	1 1	hold	00010	00001	0001	1
4	1 1	hold	00001	00000	1000	1

Product = 0000 1000 = **+8** ✓. With only 4-bit A and M, −M would wrap round to −8 and give the wrong sign. The 32-bit version of this case, $-2^{31} \times -1 = +2^{31}$, is in the testbench, the demo program and the visualisation's "Try minimum negative × −1" button.

5.10 Cycle count and timeline

- Load + iteration 0 in the first cycle (Start rises, Busy rises immediately), iterations 1–31 in the next 31 cycles: **32 iterations**.
- One more cycle in which Busy is 0, the results are stable, and the CPU writes rd: **33 cycles** total, as measured on the board.
- Baseline signed: 64 iterations, 65 cycles. Unsigned (both versions): 32 iterations, 33 cycles. Division: 32 iterations, 33 cycles.

5.11 Why not use Booth for unsigned multiplication too?

Booth treats the top bit as **negative**. An unsigned number like 0xFFFFFFFF (4,294,967,295) would be read as −1. To use Booth for unsigned you would extend both operands by a 0 bit to 33 bits (so they're positive signed numbers) and run **33** iterations: no faster than the 32 the original unsigned multiplier already takes. So unsigned multiplication was left on the original path. (It could still save area by sharing Booth's 33-bit adder instead of the 64-bit one; that's a possible further improvement, not done here.)

5.12 What improved, and what it costs

32-bit	Original	Radix-2 Booth
Signed multiply iterations / cycles	64 / 65	**32 / 33**
Unsigned multiply	32 / 33	32 / 33 (unchanged)
Adder used for signed multiply	64-bit	**33-bit** add/subtract
Extra registers	—	A 33 + M 33 + Q 32 + Q_{-1} 1 = **99 flip-flops**
Vivado (slide 5, division placeholder)	487 LUTs / 266 FF	650 LUTs / 365 FF

A **speed-for-area trade-off**: about 2× faster signed multiplication for about a third more LUTs and flip-flops, because Booth's datapath is added alongside the template's (which unsigned multiply still uses).

5B. Radix-4 (modified) Booth — the full story

Our design is radix-2, so most quiz questions will be on radix-2 (Section 5). But "how would you make it faster?" leads straight to radix-4, and another group was told to know radix-4 by heart. This section covers it at the same depth. (We also built and tested a radix-4 version first; it's in the git history at commit `6c27ed1`, and its Verilog is shown in 5B.7.)

5B.1 The idea: two bits per cycle

Radix-2 Booth looks at one new multiplier bit per cycle and chooses from {−M, 0, +M}. Radix-4 looks at **two** new bits per cycle and chooses from **{−2M, −M, 0, +M, +2M}**, then shifts by **2**. Half as many iterations: 16 instead of 32 for 32-bit.

Why these five multiples? A pair of bits is a base-4 digit 0–3. Digit 3 would need 3M = 2M + M, which is a second addition, i.e. a second adder or an extra cycle. Booth's trick (again) is to use negative digits so that 3 never appears: "3" is written as "4 − 1", i.e. −1 here and +1 carried into the next pair. Every digit left is cheap:

- 0: nothing
- ±M: M, or ~M + 1
- ±2M: M shifted left by one bit (just wiring), or its negation

5B.2 The recoding table

Look at **overlapping groups of 3 bits** (Q_{2i+1}, Q_{2i}, Q_{2i-1}), stepping by 2, with $Q_{-1} = 0$. Group i gives

$$\mathbf{d_i = -2 \cdot Q_{2i+1} + Q_{2i} + Q_{2i-1}}$$

$Q_{2i+1}\ Q_{2i}\ Q_{2i-1}$	d_i	Add to A	In words
0 0 0	0	nothing	inside a run of 0s
0 0 1	+1	+M	a run of 1s ended at the lower bit
0 1 0	+1	+M	a single 1
0 1 1	+2	+2M	a run of 1s ends after this pair
1 0 0	−2	−2M	a run of 1s starts at the upper bit
1 0 1	−1	−M	a run ends and another starts
1 1 0	−1	−M	a run of 1s starts at the lower bit
1 1 1	0	nothing	inside a run of 1s

How to rebuild the table in the exam without memorising it: the left bit counts **−2**, the middle and right bits count **+1** each. E.g. 101 = −2 + 0 + 1 = −1; 011 = 0 + 1 + 1 = +2.

5B.3 Why it works (the proof)

Short proof — it's two radix-2 digits merged. Radix-2 Booth uses $e_k = Q_{k-1} - Q_k$ at each bit k. Take two neighbouring radix-2 digits, at positions 2i (weight 1) and 2i+1 (weight 2), relative to 4^i:

$$e_{2i} + 2 \cdot e_{2i+1} = (Q_{2i-1} - Q_{2i}) + 2 \cdot (Q_{2i} - Q_{2i+1}) = \mathbf{-2Q_{2i+1} + Q_{2i} + Q_{2i-1}} = d_i$$

So $\Sigma\, d_i \cdot 4^i = \Sigma\, e_k \cdot 2^k$, which Section 5.3 already proved equals the multiplier's **signed** value (the top bit keeps only -2^{w-1}). Radix-4 inherits the signed handling from radix-2 for free.

Check with Q = 6 = 0110: groups $(Q_1,Q_0,Q_{-1}) = (1,0,0) \to -2$; $(Q_3,Q_2,Q_1) = (0,1,1) \to +2$. Value: $-2 \cdot 1 + 2 \cdot 4 = 6$ ✓.

Check with Q = −3 = 1101: (0,1,0) → +1; (1,1,0) → −1. Value: 1 − 4 = −3 ✓.

5B.4 The hardware

Same structure as radix-2, with four differences:

	Radix-2 (ours)	Radix-4
Bits examined per cycle	2: $\{Q_0, Q_{-1}\}$	3: $\{Q_1, Q_0, Q_{-1}\}$
Choices for the adder input	0, M, −M	0, M, −M, 2M, −2M
A width	w+1	**w+2** (2M of the most negative M needs one more bit)
Shift per cycle	1 (arithmetic)	**2** (arithmetic)
Iterations (w = 32)	32	**16**
After the shift, Q_{-1} =	the bit shifted out (old Q_0)	the **higher** of the two bits shifted out (old Q_1)

Datapath: the control decodes the 3 bits into "which multiple" (0 / M / 2M) and "negate?". A mux picks 0, M or M << 1; an XOR with the negate signal plus a carry-in of 1 makes it negative; one adder adds it to A; then {A, Q, Q_{-1}} shifts right by 2, copying A's sign into both new top bits.

Why A needs w+2 bits: in 4 bits, M can be −8, and 2M = −16 needs 5 bits; with headroom for the running sum's sign, 6. In general w+2. (Our tested version used w+3 because it also extended M by one bit for unsigned numbers, 5B.7.)

Why Q_{-1} takes the higher shifted-out bit: the next group overlaps the previous one in exactly that bit (Q_{2i+1} of this group is $Q_{2(i+1)-1}$ of the next).

5B.5 Worked example 1: −3 × 6 in 4 bits

M = −3 = **111101** (6 bits). Q = 6 = **0110**, A = 000000, Q_{-1} = 0. **Two** iterations.

Edge	{Q_1, Q_0, Q_{-1}}	digit	A + d·M	sum (6 bits)	A after (>>>2)	Q after	Q_{-1} after
1	1 0 0	−2	0 − 2(−3) = 6	000110	000001	1001	1
2	0 1 1	+2	1 + 2(−3) = −5	111011	111110	1110	0

Doing edge 1's shift by hand: {A, Q, Q_{-1}} = 000110 0110 0 (11 bits). Shift right by **2**, copying the sign (0) twice: 00 000110 0110 0 → drop the two lowest bits → **000001 1001 1**. The two low bits of the sum (10) went into the top of Q; the two used multiplier bits (Q_1Q_0 = 10) fell out, and the **higher** one (1) is the new Q_{-1}.

Product = {A[3:0], Q} = **1110 1110 = 0xEE = −18** ✓.

5B.6 Worked example 2: the same multiplication in radix-2 and radix-4 (25 × −14, 8 bits)

This shows that **one radix-4 step = two radix-2 steps**. M = 25, Q = −14 = 1111 0010. Product −350.

Radix-2 (8 iterations; A 9 bits):

Edge	pair	op	A after	Q after	Q_{-1}
1	00	hold	000000000	01111001	0
2	10	A − M	111110011	10111100	1
3	01	A + M	000000110	01011110	0
4	00	hold	000000011	00101111	0
5	10	A − M	111110101	00010111	1
6	11	hold	111111010	10001011	1
7	11	hold	111111101	01000101	1
8	11	hold	111111110	10100010	1

Radix-4 (4 iterations; A 10 bits):

Edge	bits	digit	A + d·M	A after	Q after	Q_{-1}
1	100	−2	−50	1111110011	10111100	1
2	001	+1	12	0000000011	00101111	0
3	110	−1	−22	1111111010	10001011	1
4	111	0	−6	1111111110	10100010	1

Compare radix-4 edge 1 with radix-2 edge 2, radix-4 edge 2 with radix-2 edge 4, and so on: **the same values** (A is one bit wider in radix-4). The radix-4 digits are the radix-2 digits paired: (0, −1) → 0 + 2(−1) = −2; (+1, 0) → +1; (−1, 0) → −1; (0, 0) → 0. Both end with {A[7:0], Q} = 1111 1110 1010 0010 = **−350** ✓.

5B.7 Worked example 3: hard cases with the most negative M (4 bits, M = −8)

Multiplication	Edge 1	Edge 2	Product
−8 × −8 (Q = 1000)	000 → 0: A = 0	100 → −2: A = 0 − 2(−8) = **+16**	0100 0000 = **+64** ✓
−8 × 7 (Q = 0111)	110 → −1: A = +8	011 → +2: A = 2 + 2(−8) = −14	1100 1000 = **−56** ✓
−8 × −6 (Q = 1010)	100 → −2: A = +16	101 → −1: A = 4 + 8 = 12	0011 0000 = **+48** ✓

+16 needs 6 signed bits, which is why A is w+2 = 6 bits for w = 4.

5B.8 The tested Verilog (our earlier version, commit `6c27ed1`)

This version also handled **unsigned** numbers with the same hardware. It extended both operands by one bit (the sign for signed, 0 for unsigned), so they became (w+1)-bit signed numbers, and padded the multiplier to w+2 bits to make the bit count even. That's 17 iterations / **18 cycles** for both signed and unsigned at 32 bits, and A was w+3 bits.

```
booth_bits = { n_shifted_op2[1:0], n_booth_prev } ;          // {Q1, Q0, Q-1}
case( booth_bits )
    3'b001, 3'b010, 3'b101, 3'b110: booth_pp = M  (sign-extended) ;
    3'b011, 3'b100:                 booth_pp = 2M (M shifted left 1) ;
    default:                        booth_pp = 0 ;
endcase
booth_neg = booth_bits[2] & ~(booth_bits[1] & booth_bits[0]) ;  // 100, 101, 110 are negative
booth_sum = n_booth_acc + (booth_pp ^ {(width+3){booth_neg}}) + booth_neg ;  // one adder: +pp or
~pp+1

n_booth_prev  = n_shifted_op2[1] ;                                    // the higher shifted-out bit
n_shifted_op2 = { booth_sum[1:0], n_shifted_op2[width+1:2] } ;        // 2 product bits in, 2 used
bits out
n_booth_acc   = { {2{booth_sum[width+2]}}, booth_sum[width+2:2] } ;  // >>> 2: copy the sign twice
```

Line by line: build the 3-bit group; pick |d|·M (M or 2M or 0); `booth_neg` is 1 exactly for 100, 101, 110 (left bit 1, but not 111); invert-and-add-1 makes it negative inside the single adder; then shift by 2, saving Q_1 as the new Q_{-1}. It passed the same 76,083-check testbench as the final design.

5B.9 Radix-2 vs radix-4 vs baseline (32-bit signed multiply)

	Baseline	Radix-2 Booth (ours)	Radix-4 Booth
Multiplier bits handled per cycle	1 (of a 64-bit sign extension)	1	2
Iterations / cycles incl. write-back	64 / 65	**32 / 33**	16 / 17 (18 in our unsigned-capable version)
Adder input choices	0, M	0, ±M	0, ±M, ±2M
Control	1 bit	2 bits → add / sub / hold	3 bits → mux (0/M/2M) + negate
A width	64-bit sum	33	34 (35 in our version)
Shift	multiplicand ← 1, multiplier → 1	{A,Q,Q_{-1}} >>> 1	{A,Q,Q_{-1}} >>> 2
Complexity	lowest	low	a little more (wider mux, 3-bit decode)

Going further: radix-8 handles 3 bits per cycle but needs digits up to ±4, including **±3M**, which isn't a shift: it needs an extra adder or a precomputed 3M register. That's why radix-4 is the usual sweet spot.

5B.10 Unsigned and odd widths in radix-4

Booth (any radix) treats the top bit as negative, so for unsigned numbers you first add a 0 on top, which makes the bit count odd. Radix-4 consumes bits in pairs, so you pad by one more bit (a copy of the new top bit) to make it even. That's why our earlier version used w+2 = 34 multiplier bits → 17 iterations.

5C. Practice problems (with answers)

Do these on paper, then check them in `booth_visualisation.html` (radix-2) or against the answers.

Radix-2, 4-bit:

Problem 1 — 5 × −3. M = 00101, −M = 11011, Q = 1101.

Edge	pair	op	A before shift	A after	Q after	Q_{-1}
1	10	A − M	11011	11101	1110	1
2	01	A + M	00010	00001	0111	0
3	10	A − M	11100	11110	0011	1
4	11	hold	11110	11111	0001	1

Product 1111 0001 = **−15**.

Problem 2 — −4 × −4. M = 11100, −M = 00100, Q = 1100.

Edge	pair	op	A before shift	A after	Q after	Q_{-1}
1	00	hold	00000	00000	0110	0
2	00	hold	00000	00000	0011	0
3	10	A − M	00100	00010	0001	1
4	11	hold	00010	00001	0000	1

Product 0001 0000 = **+16**.

Problem 3 — 7 × −3. Edges: 10 A − M → 11100 / 1110 / 1; 01 A + M → 00001 / 1111 / 0; 10 A − M → 11101 / 0111 / 1; 11 hold → 11110 / 1011 / 1. Product 1110 1011 = **−21**.

Radix-4, 4-bit (A 6 bits):

Problem 4 — −5 × 7. M = 111011, Q = 0111.

Edge	bits	digit	A + d·M	A after	Q after	Q_{-1}
1	110	−1	0 + 5 = 5	000001	0101	1
2	011	+2	1 − 10 = −9	111101	1101	0

Product 1101 1101 = **−35**.

Problem 5 — 6 × 6. M = 000110, Q = 0110.

Edge	bits	digit	A + d·M	A after	Q after	Q_{-1}
1	100	−2	−12	111101	0001	1
2	011	+2	−3 + 12 = 9	000010	0100	0

Product 0010 0100 = **+36**.

Problem 6 — 7 × −3. Edges: 010 → +1: A = 7 → 000001 / 1111 / 0; 110 → −1: A = 1 − 7 = −6 → 111110 / 1011 / 1. Product 1110 1011 = **−21** (2 edges, against 4 in radix-2 for the same problem).

Quick recoding drills (answers in brackets): - Radix-2 digits of 0111 (from the right): [−1, 0, 0, +1 → −1 + 8 = 7] - Radix-2 digits of 1000: [0, 0, 0, −1 → −8] - Radix-4 digits of 0111: [(1,1,0) → −1; (0,1,1) → +2 → −1 + 8 = 7] - Radix-4 digits of 1000: [(0,0,0) → 0; (1,0,0) → −2 → −8] - Radix-4 digits of 0101: [(0,1,0) → +1; (0,1,0) → +1 → 1 + 4 = 5]

5D. Other enhancement options (for "why Booth?" questions)

Option	What it does	Effect
Radix-2 Booth (ours)	Signed multiplier recoded to {−1, 0, +1}	Signed mul 64 → 32 iterations; more area
Radix-4 Booth	2 bits per cycle, digits {−2…+2}	16 iterations
Sharing one adder between mul and div	One adder/subtractor serves both units	Less area, same speed
Non-restoring division	Never "restores": if R went negative, add D in the next step instead of subtracting	Removes the restore mux; same 32 cycles
SRT / radix-4 division	2 quotient bits per cycle	~16 cycles, much more complex
Early termination	Stop when the remaining multiplier bits are all 0s (or all 1s)	Fewer cycles for small multipliers; variable latency
`*` operator / DSP	Single-cycle multiplier in DSP blocks	**Forbidden** by the manual

Why radix-2 Booth: it directly attacks the baseline's biggest waste (64 iterations for signed numbers, because of the 64-bit sign extension), it's simple enough to prove and verify exhaustively, and it gives a clean, measurable 2× speed-up.

---

6. Task 2: putting MCycle into the CPU

6.1 How M instructions are encoded

M instructions use the same opcode as `add`/`sub`, **0110011 (R-type)**. They're told apart by **funct7 = 0000001**, which is 0000000 for add and 0100000 for sub. funct3 then picks the operation:

funct3	Instruction	Meaning	MCycleOp	Which result
000	mul	low 32 bits of a × b	00	Result1
001	mulh	high 32 bits, signed × signed	00	Result2
010	mulhsu	high 32 bits, signed × unsigned	(not supported, NOP)	—
011	mulhu	high 32 bits, unsigned × unsigned	01	Result2
100	div	signed quotient	10	Result1
101	divu	unsigned quotient	11	Result1
110	rem	signed remainder	10	Result2
111	remu	unsigned remainder	11	Result2

Why `mul` doesn't care about signed or unsigned: the low 32 bits of a product are the same either way, because of the modulo argument in 3.3. Only the *high* word differs, which is why there are separate `mulh` and `mulhu`.

Why rem/mulh were free to add: MCycle always computes both words (product low and high, or quotient and remainder). Supporting them only needed one more mux select bit. `mulhsu` would need MCycle to treat the two operands differently, so it was left out; the manual says these extras are optional.

6.2 Decoder changes

The original Decoder looked only at `Funct7[5]` for R-type, so `mul` (funct7 = 0000001) would have decoded as `add`. Now:

```
OP_DPREG: begin
    if( Funct7 == FUNCT7_M ) begin       // 0000001: an M instruction
        case( Funct3 ) ... set MCycleOp and MResultSel per the table ... endcase
        if( Funct3 != 3'b010 ) begin     // everything except mulhsu
            RegWrite  = 1 ;
            MStart    = 1 ;              // start MCycle
            MCycleSel = 1 ;              // write MCycle's result, not the ALU's
        end
    end
    else begin                           // normal add/sub/and/...
        RegWrite = 1 ; ALUControl = {Funct3, Funct7[5]} ;
    end
end
```

The four new outputs:

- **`MStart`**: connects to MCycle's Start.
- **`MCycleOp`**: which of the four operations.
- **`MCycleSel`**: the register write data comes from MCycle.
- **`MResultSel`**: Result1 (low word or quotient) versus Result2 (high word or remainder).

All four default to 0, so no other instruction can start MCycle by accident. Unsupported or illegal encodings write nothing, the same principle as in the Assignment 2 decoder.

6.3 The datapath in RV.v

```
MCycle #(.width(32)) MCycle1( .CLK(CLK), .RESET(RESET), .Start(MStart), .MCycleOp(MCycleOp),
                              .Operand1(RD1), .Operand2(RD2),
                              .Result1(MCycleResult1), .Result2(MCycleResult2), .Busy(Busy) ) ;

assign MCycleResult = MResultSel ? MCycleResult2 : MCycleResult1 ;
assign ExecResult   = MCycleSel  ? MCycleResult  : ALUResult ;   // the new multiplexer
assign Result       = MemtoReg   ? ReadData      : ExecResult ;
```

- The operands come straight from the register file: RD1 = rs1, RD2 = rs2, just like the ALU's register operands.
- `width` is set to 32 here. The default of 4 is only for unit testing.
- The new multiplexer sits between the ALU and the existing memory mux. The manual asked for "a multiplexer and a control signal to combine the outputs from the ALU and MCycle", and this is it.

6.4 The stall: `WE_PC = ~Busy`

In a single-cycle CPU, every instruction must normally finish in one clock. A `mul` takes 33, so the CPU has to **freeze** until MCycle is done.

```
assign WE_PC = ~Busy ;
```

- While Busy = 1, the program counter doesn't update.
- Because the PC doesn't change, instruction memory keeps outputting the **same instruction**.
- So the Decoder keeps generating the same control signals, and Start stays at 1.
- Everything downstream sees the `mul` instruction, cycle after cycle, until Busy falls.

This is why Busy must be combinational (Section 2.2). In the very first cycle of the `mul`, Busy must already be 1 before the clock edge, or the PC would move on to the next instruction immediately.

6.5 Holding the register write: `WE = RegWrite & ~Busy`

While the CPU is stalled, the Decoder still says RegWrite = 1, because the instruction is still `mul`. Without the gating, the register file would be written every stall cycle with whatever partial value MCycle currently outputs. So:

```
assign WE = RegWrite & ~Busy ;
```

The register is written **only once**, in the cycle where Busy falls. In that cycle:

- MCycle's results are already final (Section 2.5),
- the register file writes rd at the clock edge,
- and the PC updates to the next instruction, at the same edge.

That's exactly what a single-cycle instruction does, just later.

Is the gating strictly necessary? Here, functionally, not quite: MCycle captured its operands at the start, and the last write would be correct anyway. It's still the right design: no intermediate garbage ever appears in a register, and it's the behaviour you'll need when you pipeline the CPU in Assignment 4.

6.6 Why capturing the operands at the start matters

Take `div a6, a6, a1`, where rd is the same register as rs1. If writes weren't held off, *and* MCycle re-read Operand1 every cycle, a partial result written into a6 would change the dividend mid-calculation

and the answer would be wrong. MCycle reads its operands only in the start cycle (Section 2.4), and the register isn't written until the end, so this works. The demo program tests exactly this case (rd = rs1 for `div`, rd = rs2 for `mul`), and the testbench additionally changes the operand inputs randomly while MCycle is busy.

6.7 Timeline of `mul t0, a0, a1` followed by `mulh t1, a0, a1`

Cycle	PC points to	MCycle state	Busy	WE_PC	Register write
1	mul	IDLE → starting	1	0	no
2–32	mul	COMPUTING	1	0	no
33	mul	COMPUTING, done	0	1	**t0 written**
34	mulh	IDLE, Start = 1	1	0	no (a new operation starts)
…	mulh	…	…	…	…
66	mulh	done	0	1	**t1 written**

In cycle 33, Start is still 1 (it's still the `mul`), but the state machine goes COMPUTING + done → IDLE regardless of Start, so it doesn't restart by mistake. In cycle 34, the new instruction is a `mulh` and Start is 1 again, so a fresh operation begins. Back-to-back M instructions therefore work with no gap, which the program's self-check loop does seven times in a row.

6.8 Wrapper.v

The new program is 128 instructions and about 161 words of data (test inputs, expected results, display buffer). The default memories were 128 words each, so `IROM_DEPTH_BITS` and `DMEM_DEPTH_BITS` were raised from 9 to 10, giving 256 words each. Wrapper.v's own comments say to change these if your program is larger. Nothing else in the Wrapper changed.

---

7. The demo program (`test.asm`)

7.1 The data

- **`cases`:** 16 pairs (a, b), chosen to hit corner cases: all sign combinations, the most negative number, overflow, zero, divide by zero, large values.
- **`expected`:** 7 results per case (mul, mulh, mulhu, div, divu, rem, remu), 112 words. They were computed in Python with the RISC-V rules, then checked independently: when RARS runs the program, its own M instructions agree with every entry and the program's self-check gives `status = 0xFF`.

Case	a	b	Why
0–3	±7	±3	all sign combinations
4	0x80000000	−1	div overflow (result -2^{31}, remainder 0)
5	0x80000000	1	most negative number
6	0x7FFFFFFF	0x7FFFFFFF	largest positive product
7	−1	−1	signed 1 vs unsigned huge
8 / 9	0 / 5	5 / 0	zero dividend / zero divisor
10, 11	±123456789	1000	ordinary multi-digit
12	0x12345678	0x9ABCDEF0	large mixed
13	−2	2	divu of a "negative" number
14	1	0x80000000	divisor larger than dividend
15	0xDEADBEEF	0x00010001	pattern

7.2 Part 1: the self-check

For each case, the program runs all seven M instructions back to back, compares each with the table, and clears that instruction's LED bit on any mismatch:

```
beq  t0, a3, ok_mul
andi s3, s3, -2       # -2 = ...11111110: clears bit 0 (LED0)
ok_mul:
```

`andi` with −2, −3, −5, −9, ... clears bits 0, 1, 2, 3, ... respectively, because −2 is all ones except bit 0, −3 is all ones except bit 1, and so on.

After all 16 cases, LED7 is set if all seven flags survived, and the value is written to the LEDs.

7.3 Part 2: measuring cycles

```
lw  t0, CYCLECOUNT_OFF(s0)    # read the counter
lw  t1, CYCLECOUNT_OFF(s0)    # read again immediately
sub s4, t1, t0                # = 1: the cost of the measuring itself
lw  t0, CYCLECOUNT_OFF(s0)
mul t2, a0, a1
lw  t1, CYCLECOUNT_OFF(s0)
sub s5, t1, t0                # = 1 + 33 = 34
sub s5, s5, s4                # = 33: the mul alone
```

The cycle counter increments every CPU clock.

- Two back-to-back `lw` reads differ by 1 cycle, because the second `lw` executes 1 cycle after the first.
- With a `mul` in between, they differ by 1 + 33.
- Subtracting the "empty" measurement removes the measuring overhead, leaving exactly the M instruction's cycles.

This is why the board shows 0x21 = 33 for both: it's the hardware measuring itself. With the original signed multiplier (BOOTH = 0) the mul would measure 0x41 = 65, which is the Task 3 speed-up shown on the real FPGA.

7.4 Part 3: the display loop

- **SW15 = 0 (browse):**
  - Reads the case number from SW3–0, loads that case's a and b, and **recomputes** all seven results with M instructions (so the display is live hardware, not the table).
  - Stores them in a 16-word `view` array.
  - Shows `view[SW7–4]`. The array approach avoids a long chain of branches.
- **SW15 = 1 (own operands):**
  - `slli a0, t0, 17` then `srai a0, a0, 25` extracts SW14–8 as a signed 7-bit number. The left shift puts bit 14 at bit 31; the arithmetic right shift brings it back down while copying the sign: sign extension (Fact 2).
  - Then the buttons pick which operation to show.

7.5 What the board does

The bitstream turns the FPGA into the whole system: the RISC-V CPU with MCycle, instruction memory pre-loaded with `test.asm`, data memory pre-loaded with the tables, and the peripherals. The CPU clock is 100 MHz ÷ 2^5 = **3.125 MHz** (`CLK_DIV_BITS = 5` in TOP); the start-up self-check takes about 5,200 cycles (about 1.7 ms), so it looks instant.

- **After programming or BTND (reset):** LED7–0 all on (0xFF) if the self-check passed. LED0 mul, LED1 mulh, LED2 mulhu, LED3 div, LED4 divu, LED5 rem, LED6 remu, LED7 all.
- **LED15–9** show the PC (too fast to read); **LED8** is the CPU clock.
- **SW15 = 0:** SW3–0 = test case; SW7–4 = 0 a, 1 b, 2 mul, 3 mulh, 4 mulhu, 5 div, 6 divu, 7 rem, 8 remu, 9 mul cycles (`21` = 33), A div cycles (`21` = 33), B LED value.
- **SW15 = 1:** SW14–8 = a, SW6–0 = b (7-bit signed, −64…63). Shows a × b; hold BTNL for a ÷ b, BTNC for a rem b, BTNR for divu.
- **BTNU** pauses the CPU clock; **BTND** resets and reruns the self-check.

Suggested demo: point at the 8 LEDs; show case 4 view 5 ($-2^{31} \div -1$ = `80000000`); show view 9 (mul: 33 cycles, against 65 for the original signed multiplier); let the TA enter their own numbers with SW15 up.

---

8. How everything was tested

8.1 Self-checking testbenches

The original `test_MCycle.v` only applied a few inputs, and you had to read the waveforms to decide whether the answers were right. A **self-checking** testbench computes the expected answer itself and prints PASS or FAIL, so you can run thousands of tests without looking at a single waveform.

The expected answers come from Verilog's own `*`, `/` and `%` operators. Those are fine in a testbench because testbenches are never synthesised. It's an *independent* reference: a completely different implementation from the hardware being tested.

8.2 What it checks

- **Exhaustively at widths 4 and 6.** Every possible pair of operands, for all four operations. The algorithms are identical at every width (only the parameter changes), so proving them correct for *every* input at small widths is strong evidence for width 32.

- **At width 32:** a 10 × 10 grid of corner values (0, ±1, ±2, most negative, most positive, alternating bit patterns) plus 3,000 random pairs per operation.
- **Exact cycle counts.** Results arriving at the wrong time would break the CPU.
- **Busy rising in the same cycle as Start.** This is the property the stall depends on (6.4).
- **The ways the CPU can drive it:**
  - Start held high back to back (consecutive M instructions).
  - Start dropped for a cycle, checking the results are held while idle.
  - Operands changed randomly while Busy, checking the operand capture of 6.6.
- **A reset in the middle of an operation.**
- **Both versions,** BOOTH = 1 and 0, for the comparison.

Result: 76,083 checks pass.

Mutation testing. A testbench that passes proves little if it would also pass a broken design. So the design was deliberately broken in many different ways: wrong remainder sign, a Booth add on the wrong bit pair, a logical instead of arithmetic shift, M not sign-extended, the product taken from the wrong bits, one iteration short or long, operands not latched, and so on. The testbench failed every time, which shows it really can catch those bugs.

8.3 Post-synthesis simulation (Task 3 requirement)

- **Behavioural simulation** runs your Verilog as written.
- **Post-synthesis simulation** runs the netlist of LUTs and flip-flops that synthesis produced from it.

They can differ: a missing signal in a sensitivity list, an unintended latch, or reliance on initial values can make simulation and real hardware disagree. Running the same tests on the netlist proves the synthesised circuit really does what you simulated.

There's one complication. A synthesised netlist has no parameters left, because width and BOOTH were fixed during synthesis. So `test_MCycle_postsynth` instantiates MCycle *without* overriding parameters, using a Verilog `generate if` to choose between the two ways of instantiating. It was run on gate-level netlists of both versions; both pass, with signed multiply taking 33 and 65 cycles respectively.

Doing it in Vivado:

1. Set `MCycle` as top under Design Sources.
2. *Settings* → *Synthesis* → *More Options*: `-generic width=32 -generic BOOTH=1`.
3. Run Synthesis; note LUTs and FFs from *Report Utilization*.
4. Set `test_MCycle_postsynth` as the simulation top (`localparam BOOTH = 1`), run *Post-Synthesis Functional Simulation*, then `run all`.
5. Repeat with `BOOTH=0` and `localparam BOOTH = 0`.
6. Afterwards, remove the `-generic` options and set `TOP` / `test_Wrapper` back as tops.

8.4 CPU level

- **`test_Wrapper.v`** (229 checks) runs the real program on the whole CPU and checks:
  - the LEDs reach 0xFF,
  - every one of the 12 views for all 16 cases,
  - the measured cycle counts,
  - the own-operand mode with each button.

- `cpu_check.py` is the Assignment 2 test. It re-checks every RV32I instruction to prove that adding MCycle broke nothing; all 6,703 checks pass. Only MCycle.v was added to its file list and the M instructions to its model of the Decoder.

8.5 Vivado messages you can ignore

When MCycle is synthesised as top: the `TOP_Nexys4DDR.xdc` warnings (`No ports matched 'CLK_undiv'`, `No pins matched 'RX_MSF1_reg/C'`, `create_clock: No valid object(s)`) appear because the constraints are written for `TOP`'s ports; disable the XDC for a completely clean run, then re-enable it. The incremental-synthesis checkpoint warnings (`TOP.dcp is not suitable…`) are harmless; you can turn incremental synthesis off. `Too many words specified in data file AA_DMEM.mem` is normal (RARS dumps more data than DMEM holds).

8B. Task 4: the visualisation, and the division walkthrough

8B.1 What Task 4 asks for

A **single HTML file** (HTML + CSS + JavaScript), made with AI help, that teaches the Task 3 enhancement to someone who doesn't know it: enter operands, step cycle by cycle, and see the registers and datapath elements, the operation at each step, how the state changes, and the cycle count. It must match **our** implementation, not a generic one, and you must submit the **complete prompt history** (plus any skills/Markdown files given to the AI). You must understand the algorithm, not the JavaScript.

8B.2 Our page: `booth_visualisation.html`

Open it in any browser (no internet needed). From top to bottom:

Part	What it shows	How it maps to the hardware
Inputs	Width (4 / 8 / 32 bits), multiplicand M, multiplier Q (decimal, or hex like `0xFFFFFFFD`), Load	Operand1 / Operand2 and the `width` parameter
Buttons	**Next clock edge**, Run/Pause, Reset trace, "Try minimum negative × −1", Export trace CSV	Each "Next clock edge" = one rising clock edge = one iteration
Stats	Booth edges completed (e.g. 2 / 4); template vs Booth iterations / cycles (e.g. 64 / 65 vs 32 / 33); State / Busy	`count`, the latency, and Busy
Registers	M, A, Q, Q_{-1} bit by bit; **yellow** = changed at this edge; **teal outline** = the two bits {Q_0, Q_{-1}} that decide the next edge	`booth_M`, `booth_A`, `booth_Q`, `booth_Qminus1`
Operation box	e.g. "Edge 2: A + M, then arithmetic right shift; read 01; A before shift = 11110"	The case statement and the shift
Decision table	The 00 / 01 / 10 / 11 table with the row for the next edge highlighted	The `case` in MCycle.v lines 205–209
Datapath	M → (+M / −M = ~M + 1 / 0) → select → (w+1)-bit adder ← A; → arithmetic shift right → A, Q, Q_{-1}; control reads {Q_0, Q_{-1}}. The path used at this edge is teal, idle parts grey, with live hex values on the wires	The whole Booth datapath, one edge at a time
What happens at one edge	Read → add/sub/hold → shift and register, as one register update	The single combinational pass (Section 2.3)
Trace table	Every edge: pair read, operation, A before shift, A, Q, Q_{-1} after	The register history
Final answer	e.g. "−3 × 5 = −15 · product bits 11110001"	{A[w−1:0], Q}

It matches the RTL exactly: its model was compared with a Verilog simulation of MCycle.v; A, Q and Q_{-1} after every edge are identical (4, 8 and 32 bits, including $-2^{31} \times -1$), and its products are correct for every 4- and 6-bit operand pair.

Using it in the demo: load −3 × 5 at 4 bits and press "Next clock edge" four times while saying what each edge does (Section 5.7). Then load the TA's numbers. "Try minimum negative × −1" shows why A and M need the extra bit.

8B.3 The prompt history (`prompt_history.md`)

Part 1 is Hin Lok's original prompts (to be pasted in from his PROMPT_HISTORY.md, plus his GUIDE.md); Part 2 records the enhancement prompt (datapath diagram, cycle counts, hex input). The submission must contain the **complete** history, so make sure Part 1 is filled in.

8B.4 Qi Ye's division walkthrough (`docs/CG3207_Task1_Division_Dataflow.html`)

A study aid (not a deliverable). It follows one 4-bit unsigned division, **13 ÷ 3**, **signal by signal** through MCycle.v. For each step it shows the stored registers (`temp_sum`, `count`, `done`, `state`, …) next to their proposed next values (`n_temp_sum`, …), highlights what differs, and shows the exact MCycle.v lines responsible. It makes the "registers vs `n_` variables" idea (Section 2.3) concrete: the `n_` values are worked out step by step within a cycle, and the registers only change at the rising edge. It ends with the write-back cycle, when Busy falls and the CPU writes the result. The line numbers match the final MCycle.v.

---

9. Questions you're likely to be asked (with answers)

Booth radix-2 (expect most questions here)

Explain Booth's algorithm in one sentence. Scan the multiplier from the LSB, looking at each bit and the one to its right: at the start of a run of 1s (10) subtract M, at the end of a run (01) add M, otherwise do nothing; shift {A, Q, Q_{-1}} right arithmetically after each step; after w steps {A, Q} is the product.

What is Q_{-1} for? It remembers the bit that was just shifted out of Q, so that each step can see two adjacent bits and detect where a run of 1s starts or ends. It starts at 0 (an imaginary bit to the right of the LSB).

Why does 10 mean subtract and 01 mean add? A run of 1s from bit i to bit j equals $2^{j+1} - 2^{i}$. The 10 pair is at bit i (the run starts: $-M \cdot 2^{i}$), and the 01 pair is at bit j+1 (the run has ended: $+M \cdot 2^{j+1}$).

Why does Booth work for negative multipliers? The digit is $d_i = Q_{i-1} - Q_i$. Summing $d_i \cdot 2^{i}$, every bit gets its normal weight except the top bit, which keeps only -2^{w-1}: exactly the two's complement sign weight (Section 5.3).

Why does the baseline take 64 iterations for signed but Booth 32? The baseline can only add positive multiples, so it sign-extends to 64 bits and processes all 64 bits. Booth handles the sign bit's negative weight in the recoding, so it only processes the 32 real bits.

Why is A 33 bits? So that −M fits when $M = -2^{31}$, and so the running sum doesn't overflow before the shift. Bit 32 is discarded at the end.

Why must the shift be arithmetic? A holds a signed partial product. Shifting right halves it, and halving a negative number must keep it negative, so the sign bit is copied in.

Where does the product end up? {A[31:0], Q}. Q starts as the multiplier; each shift moves one product bit from A into Q and one used multiplier bit out into Q_{-1}.

How is A − M implemented? A + ~M + 1 (invert and add 1). Synthesis does this from `n_booth_A - n_booth_M`, using the adder's carry-in, so add and subtract share one adder.

What happens for 00 and 11? Nothing is added; A only shifts. 00 is inside a run of 0s, 11 inside a run of 1s.

Walk me through −3 × 5 / show me edge 2. Section 5.7. Edge 2: pair 01 → A = 1 + (−3) = −2 = 11110 → shift {11110, 1010, 1} → {11111, 0101, 0}.

What's the worst case for Booth? Is it ever slower? In our hardware it always takes exactly 32 iterations, whatever the operands, because every iteration does at most one add/subtract and one shift in one cycle. (In software or in variable-latency designs, alternating bits like 0101… are the worst case, because they need an add or subtract at every step. Here it doesn't matter.)

Why didn't you use Booth for unsigned multiplication? Booth reads the top bit as negative. For unsigned you'd extend to 33 bits and need 33 iterations: no faster than the existing 32 (Section 5.11).

How many extra flip-flops, and why? 99 = A 33 + M 33 + Q 32 + Q_{-1} 1.

How could you make it faster still? Radix-4 Booth: 2 bits per cycle with digits {−2, −1, 0, +1, +2}, so 16 iterations (Section 5B).

How could you make it smaller? Share Booth's 33-bit adder with unsigned multiply (extend unsigned operands by a 0 bit) and remove the 64-bit shift-and-add, or share one adder between multiply and divide.

Booth radix-4

What are the radix-4 digits and why? {−2, −1, 0, +1, +2}, from overlapping 3-bit groups: d = $-2 \cdot Q_{2i+1} + Q_{2i} + Q_{2i-1}$. All are cheap: 2M is a shift, negatives are invert + 1. Digit 3 is avoided because 3M needs an extra adder.

Why overlapping groups of 3? Each digit needs the previous pair's top bit (Q_{2i-1}) to know whether a run of 1s continues across the pair boundary, just as radix-2 needs Q_{-1}.

How many iterations, and what shift? w/2 = 16 for 32-bit, shifting right by 2 each time.

Why is A w+2 bits in radix-4? ±2M needs one more bit than M, and the sum needs headroom for its sign.

What's 100 / 011 / 111 / 101? 100 → −2M; 011 → +2M; 111 → 0; 101 → −M. (Left bit −2, middle and right +1 each.)

How is radix-4 related to radix-2? Each radix-4 digit is two radix-2 digits merged: d = $e_{2i} + 2e_{2i+1} = -2Q_{2i+1} + Q_{2i} + Q_{2i-1}$. One radix-4 step gives exactly the same A and Q as two radix-2 steps (Section 5B.6), so it inherits the signed-number proof.

Why can't radix-4 use digit 3? 3M = 2M + M isn't a shift; it would need a second adder or a precomputed 3M. Booth rewrites 3 as 4 − 1 (−1 here, +1 into the next group), so only 0, ±1, ±2 are needed.

After a radix-4 step, what goes into Q_{-1}? The higher of the two bits shifted out (old Q_1), because the next group overlaps this one in that bit.

How do you negate in radix-4? Pick |d|·M (M or M shifted left 1), then XOR every bit with the negate signal and add it as the carry-in: ~pp + 1.

Recode 0111 (radix-4). (1,1,0) → −1; (0,1,1) → +2: −1 + 2·4 = 7.

How many iterations for 32-bit unsigned in radix-4? Unsigned needs a 0 on top (33 bits), padded to an even 34 bits: 17 iterations (18 cycles). That's what our first version did.

Would radix-4 cost more hardware than your radix-2? Slightly more datapath: a 5-way choice (0, ±M, ±2M) instead of 3, a 3-bit decode, and A one bit wider. The registers are about the same. Our earlier radix-4 version was actually smaller than the baseline, because it replaced the 64-bit datapath for both signed and unsigned instead of adding beside it.

What about radix-8? Digits up to ±4 include ±3M, which needs an extra adder or a precomputed 3M. That's why radix-4 is the usual choice.

Division (Task 1)

Explain restoring division. {R, Q} = {0, dividend}. Each cycle: shift {R, Q} left by 1, compute R − D; if ≥ 0 keep it and set the new quotient bit to 1, else keep the old R ("restore") and set 0. After 32 cycles Q is the quotient and R the remainder.

Why "restoring"? When R − D is negative, the subtraction is thrown away and the original R is kept.

Why does the comparison need an extra bit? After the shift, R can be up to 2D − 1, which can need one more bit than D.

How do you handle signed division? Divide the magnitudes; negate the quotient if the signs differ; give the remainder the dividend's sign. That rounds the quotient towards zero, as RISC-V requires.

What is $-2^{31} \div -1$? -2^{31}, remainder 0 (RISC-V's defined overflow result). It falls out naturally: $|-2^{31}|$ = 0x80000000 read as unsigned is 2^{31}, ÷ 1 = 0x80000000, signs equal so no negation.

What about ÷ 0? Every trial subtraction succeeds, so the quotient is all ones and the remainder is the dividend: the RISC-V-defined result. The `& (|Operand2)` term stops the quotient being negated for a negative dividend.

How could division be faster? Non-restoring division (no restore mux), or radix-4 / SRT division (2 quotient bits per cycle).

MCycle and the CPU (Task 2)

How does the CPU wait for MCycle? `WE_PC = ~Busy`: while Busy the PC holds, so the same instruction stays in the decoder and Start stays high. Busy is combinational, so it rises in the same cycle as Start, before the PC can move.

When is the result written to the register file? In the cycle in which Busy falls (`WE = RegWrite & ~Busy`): the results are final, and at that clock edge rd is written and the PC advances, like a normal single-cycle instruction.

Why gate WE with ~Busy? So partial results are never written to rd, and the write happens exactly once.

What if rd = rs1 (e.g. `div a6, a6, a1`)? MCycle copies its operands into its own registers in the first cycle and never reads them again, and rd isn't written until the end, so it works. The demo program tests this.

What if two M instructions are back to back? In the cycle Busy falls, the FSM returns to IDLE regardless of Start; the next instruction then starts a new operation in the following cycle.

How are the M instructions decoded? Opcode 0110011 (R-type) with funct7 = 0000001; funct3 picks the operation. The Decoder sets MStart, MCycleOp, MCycleSel (write MCycle's result instead of the ALU's) and MResultSel (Result1 = low word / quotient, Result2 = high word / remainder).

Why does `mul` not care about signed vs unsigned? The low 32 bits of a product are the same either way (arithmetic modulo 2^{32}). Only the high word differs: hence `mulh` vs `mulhu`.

Which instructions did you implement? mul, div, divu (required) plus mulh, mulhu, rem, remu (free, because MCycle produces both words). mulhsu is not supported (decodes as a NOP).

Verilog, synthesis and testing

What is a latch and why is it bad? Memory created by accident when a signal in an `always @(*)` block isn't assigned on every path. It makes timing unpredictable. MCycle gives every combinational signal a default at the top of the block, so there are none (slide 5: 0 latches).

Why the `n_` variables? The combinational block computes what each register will be next, step by step; the clocked block copies them all in at the edge, in one place. It lets several dependent steps (load, then the first iteration) happen in one cycle (Section 2.3).

What is a Mealy machine? One whose outputs depend on the inputs as well as the state. Busy depends on Start in the IDLE state, so it rises without waiting for a clock edge.

Behavioural vs post-synthesis simulation? Behavioural simulates the Verilog as written; post-synthesis simulates the netlist of LUTs and flip-flops synthesis produced. Running the same tests on both proves the synthesised circuit behaves like the code (Section 8.3).

How do you know the testbench is good? It's self-checking against an independent reference (Verilog's own `*`, `/`, `%`, legal in a testbench), exhaustive at 4 and 6 bits, and deliberately broken designs (mutations) all make it fail.

What do setup and hold slack mean? Setup slack: how early the slowest signal arrives before the next clock edge (must be ≥ 0). Hold slack: how long a signal stays stable after the edge (must be ≥ 0). Post-synthesis hold values are estimates without routing and are fixed in implementation.

Two's complement quickies

What is `FFFFFFEE`? −18 (invert → `00000011`, +1 → `00000012` = 18). **What is `FFFFFFF1`?** −15. **Why does divu of −6 by 3 give `55555553`?** divu reads `FFFFFFFA` as 4,294,967,290; ÷ 3 = 1,431,655,763 = 0x55555553, remainder 1.

---

10. Glossary

Term	Meaning
LUT (look-up table)	The FPGA's basic logic element: a small memory that can implement any function of up to 6 inputs
Flip-flop (FF)	A 1-bit register, updated at the clock edge
Slice	A group of LUTs and flip-flops in the FPGA
DSP block	A dedicated hardware multiplier/accumulator in the FPGA; `*` would use these
CARRY4	The FPGA's fast carry chain, used to build adders
Latch	Level-sensitive memory, usually created by mistake in combinational code
Synthesis	Turning Verilog into a netlist of LUTs and flip-flops
Implementation	Placing and routing that netlist onto the chip
Bitstream	The file that configures the FPGA with the implemented design
Behavioural simulation	Simulating the Verilog source
Post-synthesis simulation	Simulating the synthesised netlist
Setup / hold slack	Timing margins before / after the clock edge (positive = OK)
Mealy / Moore machine	FSM whose outputs depend on state and inputs / on state only
Blocking (`=`) / non-blocking (`<=`)	`=` updates immediately (used in combinational blocks), `<=` at the end of the time step (used for registers)
Two's complement	Signed number format where the top bit has negative weight
Sign extension	Widening a signed number by copying its top bit
Arithmetic shift right	Shift right copying the sign bit (signed ÷ 2, rounding down)
Latency	Number of cycles an operation takes
Radix	The number base: radix-2 processes 1 bit per step, radix-4 two bits
MCycleOp	00 signed mul, 01 unsigned mul, 10 signed div, 11 unsigned div
Busy	MCycle output: 1 while computing; stalls the CPU
RV32M	The RISC-V multiply/divide extension

---

11. Submission checklist (from the manual)

Upload before your lab slot, as `GroupXX_Monday/Friday_Asst3.zip` (your Assignment 2 folder is named "Friday39", so this is probably `Group39_Friday_Asst3.zip`). One submission per group; only these files, **not** the whole project folder; exactly the files used in the demo.

Required	Our file(s)
.v files created or modified (RTL and testbenches)	`MCycle.v`, `RV.v`, `Decoder.v`, `Wrapper.v`, `test_MCycle.v`, `test_Wrapper.v` (+ Hin Lok's testbench if the slides quote it)
.bit file	`TOP.bit` from the final build (BOOTH = 1)
.s/.asm file	`test.asm`
1–4 slides (.ppt or .pdf) on the Task 3 enhancement	the Booth slides
single-file .html visualisation	`booth_visualisation.html`
prompt history + other Markdown files used	`prompt_history.md` (Part 1 filled in), Hin Lok's `GUIDE.md`

12. One-page cheat sheet

Radix-2 Booth (signed mul, ours): A = 0 (33b), M = sign-ext multiplicand (33b), Q = multiplier (32b), $Q_{-1} = 0$. Each cycle: $\{Q_0,Q_{-1}\}$: 00/11 hold, 01 A += M, 10 A −= M; then $\{A,Q,Q_{-1}\}$ >>> 1 (copy sign). 32 iterations → product {A[31:0], Q}. 33 cycles with write-back (baseline 64 / 65). +99 FFs (33+33+32+1).

Why it works: $d_i = Q_{i-1} - Q_i$; $\Sigma\, d_i 2^i$ = signed value (top bit keeps -2^{w-1}). Runs of 1s: $2^{j+1} - 2^i$.

Radix-4: groups (Q_{2i+1}, Q_{2i}, Q_{2i-1}), left bit −2, others +1: 000/111 0, 001/010 +M, 011 +2M, 100 −2M, 101/110 −M. Choose 0/M/2M, negate by ~pp + 1; $\{A,Q,Q_{-1}\}$ >>> 2; new Q_{-1} = old Q_1; 16 iterations (17 cycles); A w+2 bits. One radix-4 step = two radix-2 steps. −3 × 6: 100 → A = +6 → 000001|1001|1; 011 → A = −5 → 111110|1110|0 → 1110 1110 = −18.

−3 × 5 (4-bit): M 11101, −M 00011, Q 0101. Edges: 10 A−M → 00001|1010|1; 01 A+M → 11111|0101|0; 10 → 00001|0010|1; 01 → 11111|0001|0. Product 1111 0001 = −15.

Restoring division: {R,Q} = {0, |dividend|}; shift left; R − D ≥ 0 ? (R = R − D, q = 1) : (restore, q = 0); 32 iterations; quotient negated if signs differ; remainder has the dividend's sign. $-2^{31} \div -1 = -2^{31}$ r 0; x ÷ 0 = −1 r x.

CPU: M instructions = opcode 0110011, funct7 0000001. WE_PC = ~Busy (stall); WE = RegWrite & ~Busy (write once, when Busy falls); result mux ALU / MCycle.

Board: LEDs 0xFF = self-check passed. SW15 = 0: SW3–0 case, SW7–4 view (2 mul … 9 mul cycles = `21`, A div cycles = `21`). SW15 = 1: SW14–8 a, SW6–0 b; × / BTNL ÷ / BTNC rem / BTNR divu. BTND reset.

Numbers: signed mul 64 → 32 iterations (65 → 33 cycles); Vivado (slide 5) 487 → 650 LUTs (+33.5%), 266 → 365 FFs (+37.2%), 0 DSP, 0 latches. Tests: 76,083 MCycle checks, 229 CPU checks, 6,703 RV32I checks.

Practice: Section 5C (radix-2 and radix-4, with answers); −7 ÷ 2 for division. Check radix-2 in the visualisation and on the board.